

Software Architecture Specification

Introduction

In competitive gaming, post-match analysis is often limited to video replays (VODs), which lack objective spatial data. Professional teams and analysts need a way to visualize positioning, rotations, and map control through coordinate-based data to identify patterns that are invisible to the naked eye.

Vantage Point will ingest game coordinate data, process spatial relationships, and provide a dashboard for analysing player positioning and team density over the course of a match.

Architectural Requirements

Architectural Patterns

1. Layered Architecture Pattern

Vantage Point is built as an N-Tier system, consisting of the following layers:

1. Frontend Layer: This mainly consists of the UI and basic functionality encoded within
2. ML and Data Processing Layer: This layer contains the KNN model and is where all intensive data processing and analysis will be done
3. API and Backend Layer: The API will communicate with the Riot Match-v5 API outside of the system to get input data. Other backend processes are also contained in this layer
4. Database Layer: Here is where the system will store and organize all users ad system data

2. Microservices Architecture Pattern

The API is built according to the microservices architecture pattern:

1. API endpoint are grouped according to functionality/ purpose

Design Patterns

Composite Design Pattern

- The match timeline data, as it is received from Riot's Match-v5 API is a string in approximate JSON format, which is translated into a composite object in the system's API.

- This makes the data easier to work with and reformat. It also allows simpler access to different parts of the data.
- This also accounts for the differing shape of match timeline data.

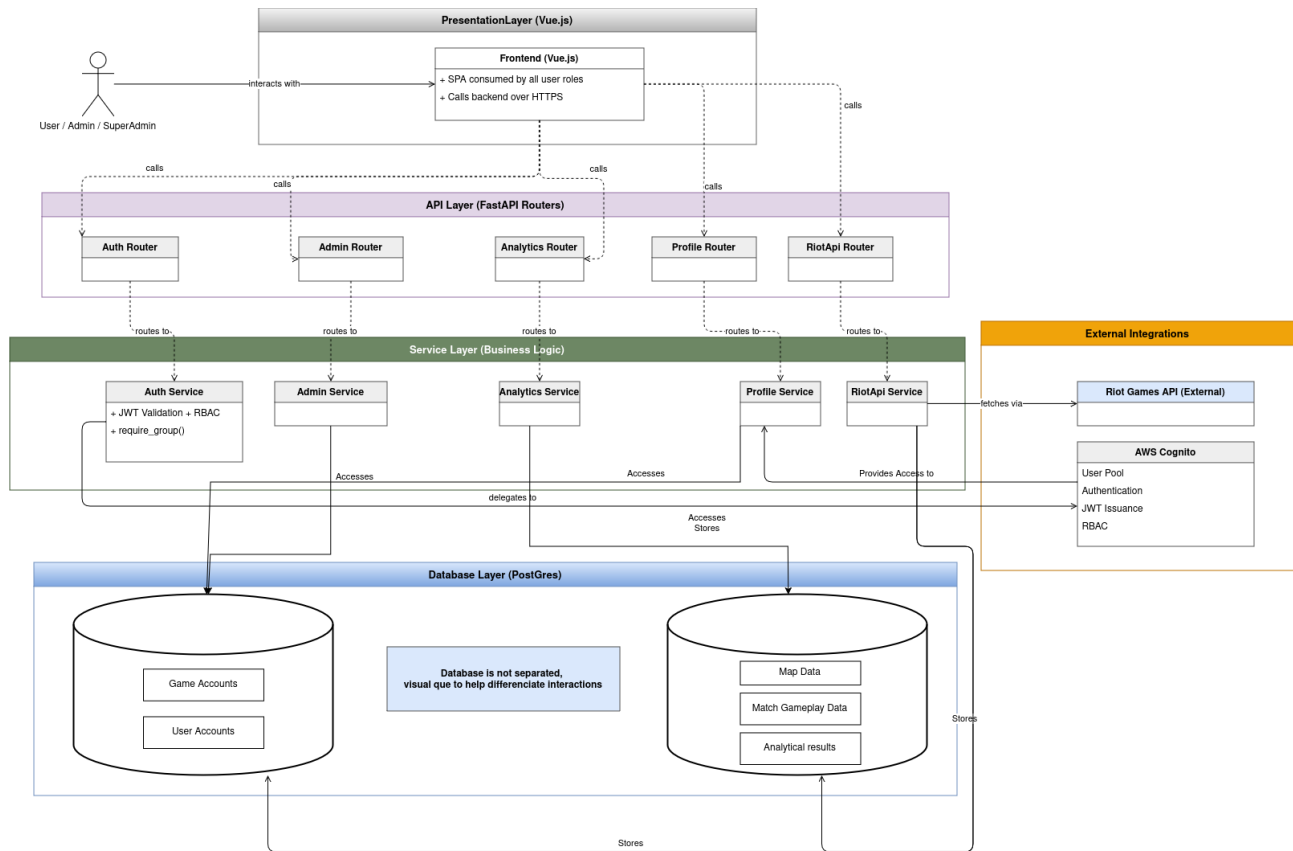
Observer Design Pattern

- The observer will be set up to watch from input data from the API and trigger requests and processing thereof at the appropriate data processing module.
- This would allow for more streamlined match data processing and faster turnaround time.

Constraints

- The system is almost entirely dependent on Riot's Match-v5 API for most of its functionality. This outside API has various restrictions and rate limits in place. This places those same limits on our system.
- The system is also reliant on the Account-v1 API for user information like puuid, which is necessary for obtaining the players match history.
- Another restriction caused by Riot's Match-v5 API is how much data can be collected, and the assumed quality thereof, in a set time frame for model training. Access to historic match data is limited to E-sports tournament matches, which may not be able to translate to regular player game-play styles.
- Map assets are only available as flat images, making somewhat more difficult to overlay map data.

Architectural Diagram



Architectural Decisions

Quality Requirement	Architectural Decision
≥ 98% uptime, critical recovery in ≤ 5 min	Containerised system on AWS server, allowance for easy manual rollback
New features and bug fixes deployed in < 2 hour	Repo structure with CI/CD pipeline, automated staging and deployment pipelines on branch merge
Unit tests coverage ≥ 65-70% of codebase	Unit test and coverage check as part of CI/CD pipeline
< 5 min downtime for deployment	Immediate container health checks on deployment, automated rollback
Secure user credentials	Use of AWS Cognito for user data management

Technology Requirements

Vantage Point utilizes a modern, data-centric stack designed to handle high-frequency coordinate data and provide predictive insights into player behaviour.

Persistence & Infrastructure:

- PostgreSQL: Stores relational data such as match histories, player profiles, and pre-calculated spatial benchmarks.
- AWS (S3 & Lambda): Utilized for serverless data fetching and cost-effective storage of raw match log files.
- Dev Containers: To eliminate environment drift, the entire stack is containerized, ensuring consistency between local development and deployment.
- AWS Cognito: For secure user login, registration, role management, and access control.

Backend & Data processing:

- FastAPI (Python): Chosen for its high-performing asynchronous execution and native support for Pydantic. It handles API rate limiting and serves as the orchestration layer for ML inference.
- Pandas & Scikit-learn: These form the core of the analytics engine. Pandas is used for cleaning and normalizing game logs, while Scikit-learn implements K-Nearest-Neighbours for snapshot comparison and Random Forest various recommendations.
- Static Analysis: To maintain the type safety often missing in Python, the project enforces strict MyPy type checking and Ruff for high-speed linting.

Frontend & Visualization:

- React: Provides a modular UI for the system, allowing consistency between pages and designs.
- D3.js: The primary spatial engine. It maps game engine coordinates to screen pixels, ensuring that player paths and events are overlaid on map assets.
- Tailwind CSS: Used for a responsive, utility-first UI, ensuring the system is usable across different screen resolutions.

API Contracts

Our API is documented using Swagger. There also exist a pdf file as a backup, available via the README.

Deployment

Deployment Requirements

Environment Parity

All branches, save for main, are deployed and run locally for testing and debugging. This is done using docker containers and local .env files.

Staging does go through the full deployment pipeline to deploy locally and allow for rollbacks. This allows use to see how the code would go through the Main branch pipeline and iron out any bugs.

Main is the only branch that gets deployed on on an AWS server, which is done by running a shell script on it via SHH.

Containerisation

The system run in two seperate Docker containers. One containing the postgres database, and another for the website, API, and other backend/frontend files.

Secrets Management

Locally, secrets are kept in three .env files, namely in the backend, frontend, and root folders. These are formatted according to .env.example files in each respective folder.

On the AWS server, secrets are managed by AWS services.

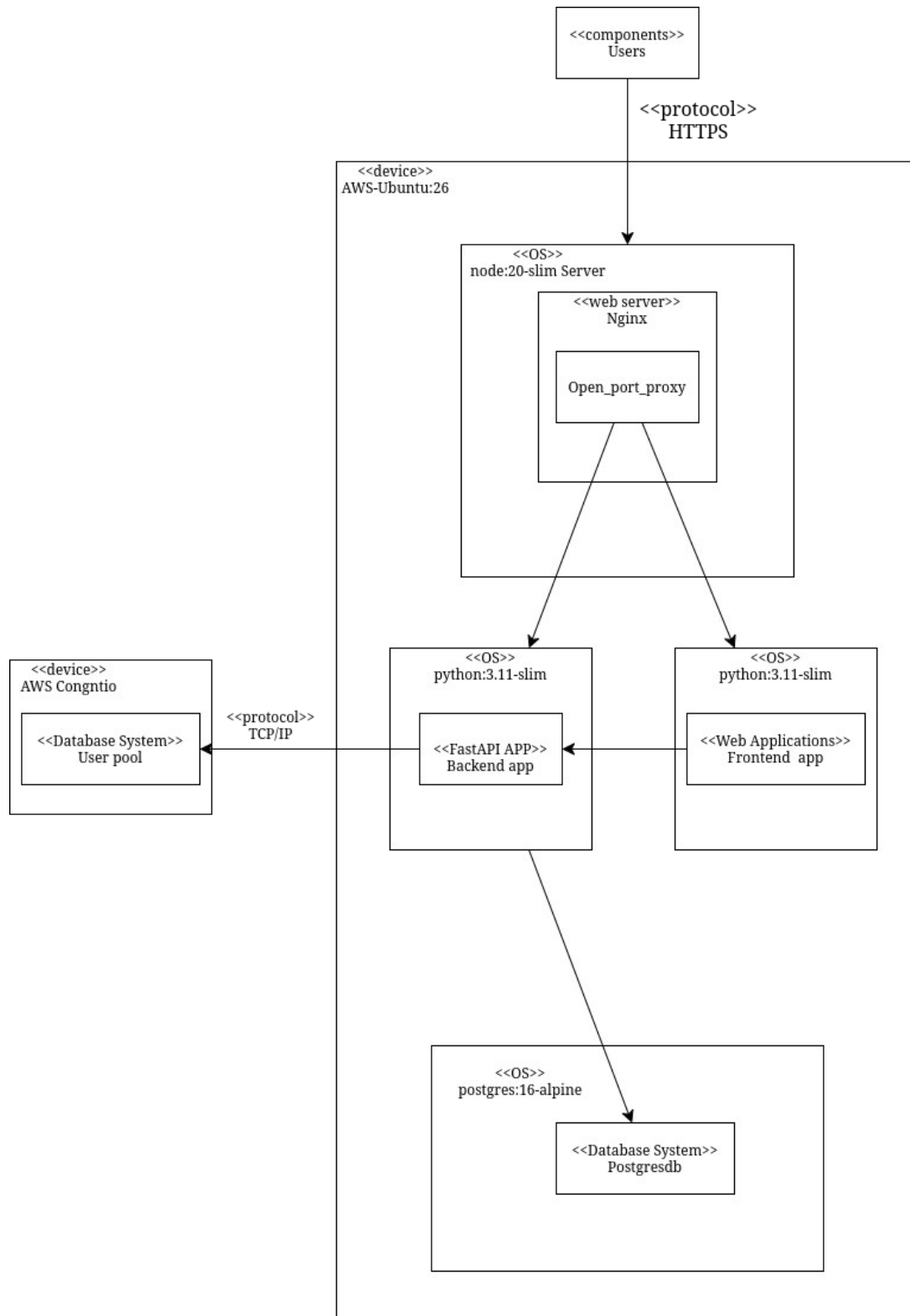
Rollback Strategy

The deployment script that is run by the CI/CD pipeline saves and prints out (to terminal) the hash of the last commit on the server's local branch. i.e. the last instance of code that was deployed and running.

Once the containers are presumed to be running a health check is done. Should this health check pass, the new version of the system is up and functional. If it should fail, and immediate rollback is done using the saved hash and the previous working version of the system is spun back up.

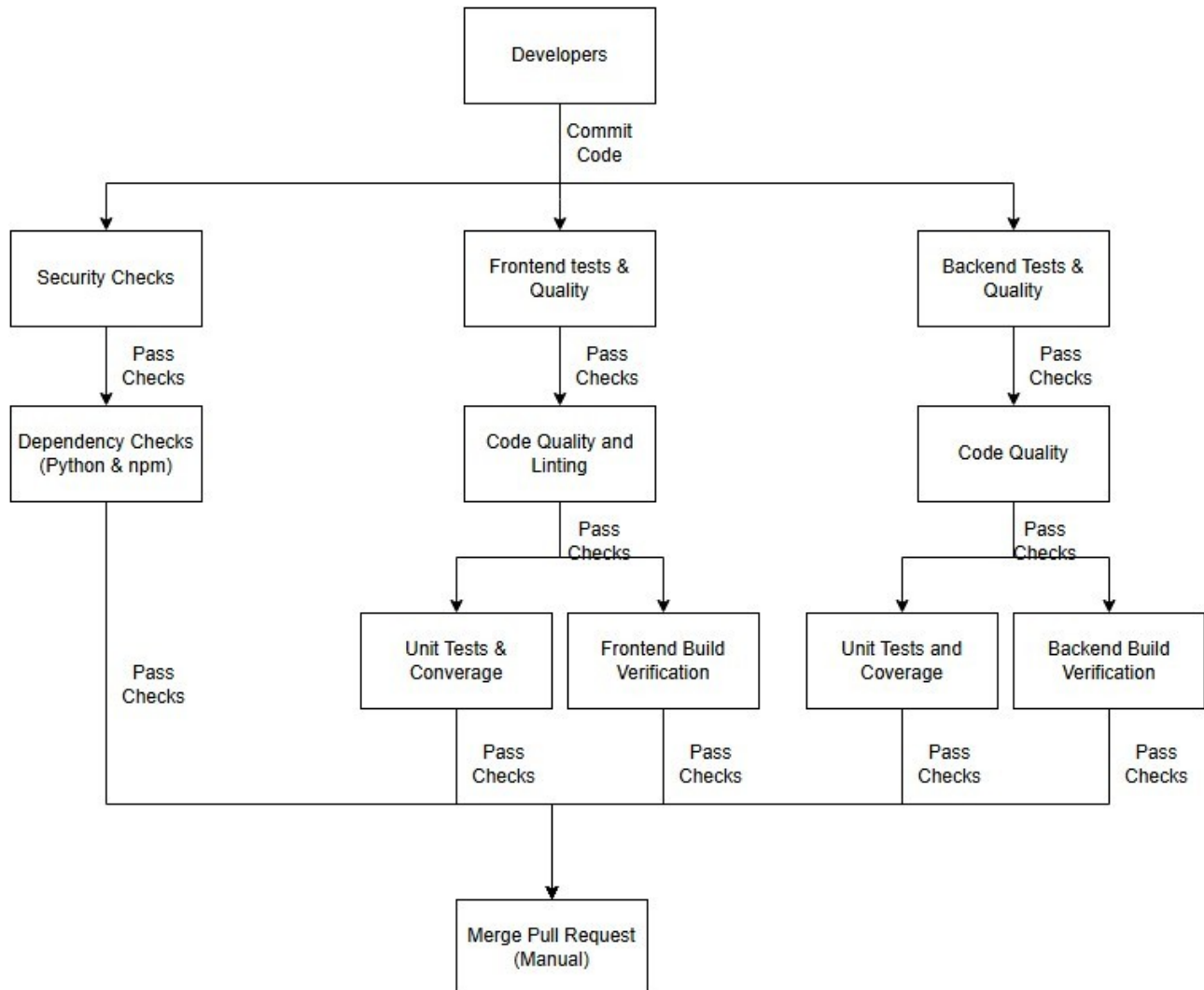
The developer who triggers the deployment script saves the terminal output of the previous hash somewhere more permanent so that, if there is a problem that was not obvious upon deployment, the server can be rolled back to the state of the previous functional commit.

Deployment Diagram



CI/CD Pipeline Diagram

When any pull request is made or any code is committed to a branch, the following process gets run to ensure that the code is functional, tested, and good quality. Backend and Frontend will only run if changes were made to the Backend and Frontend folders respectively.



The Deployment pipeline runs on the Main and Staging branches only and is activated on any push to these branches. Separate workflows exist for each of the branches, but they run the same way so they are shown on the same diagram.

*Note that Staging runs on a local server

